Distributed In-Memory Cache with Strong Consistency Guarantees

(Distributed In-Memory Cache
with Strong Consistency Guarantees)

Dominik Szczepaniak

Praca inżynierska

Promotor: dr Piotr Witkowski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

5 stycznia 2026

Abstract

Presented work is focused on the design and implementation of a distributed key-value storage system. It is designed to manage data across multiple servers with focus on data being truthful after each operation. Many modern systems like Redis, Apache Cassandra or Amazon DynamoDB focus on system being unavailable for as little time as possible. This often comes at the cost of temporary data inconsistency. This project ensures that all users see the exact same, up to date data regardless of the server that they connected to even in the event of network failures.

The system is built using an architecture that separates itself into layers. This design allows to detect server failures and automatically reorginize itself without compromising any data integrity. The system is supported with a specialized client library that optimizes communication and ensures that requests are router as efficiently as possible to the correct server.

The system was implemented by the autor in Go language, with gRPC used for internal communication and HTTP for external communication.

The correctness of solution has been tested with multiple tests. Particular emphasis was placed on many possibilies of failure, including local failures or broader network partitions. Benchmarks show that the system achieves performance comparable to industry grade standards while maintaining its goals of data consistency.

Poniższa praca opisuje zaprojektowany system rozproszony który przechowuje dane w modelu klucz-wartość. System został zaprojektowany do zarządzania danymi które zostały podzielone na wiele serwerów. Główną gwarancją i celem systemu jest to aby po każdym zapytaniu zwracał poprawną odpowiedź kosztem krótkotrwałej niedostępności. Wiele produkcyjnych systemów, takich jak Redis, Apache Cassandra czy Amazon DynamoDB, stawia na to, aby system cały czas działał, ale pozwala na to, by w małej liczbie przypadków zwrócił niepoprawne dane. Ten projekt gwarantuje, że wszyscy użytkownicy widzą takie same dane niezależnie od serwera do którego się łaczą, nawet w wypadku krótkotrwałych awarii sieci czy pojedycznych serwerów.

Architektura została zaprojektowana z podziałem na warstwy, w których każda ma określone, niezależne od siebie nawzajem cele i zadania. To pozwala na wykrywanie awarii serwerów oraz automatyczną samonaprawę systemu bez narażania integralności danych. System wspiera dedykowana biblioteka klienta, która optymalizuje komunikacje i zapewnia efektywne wysyłanie zapytań do odpowiednich serwerów.

Rozwiązanie zostało zaimplementowane w języku Go, z wykorzystaniem protokołu gRPC do komunikacji wewnętrznej oraz HTTP do komunikacji zewnętrznej.

Poprawność została zweryfikowana testami. Nacisk został przede wszystkim położony na sprawdzanie integralności danych przy różnych scenariuszach awarii systemu. Dodatkowo zostały przeprowadzone testy wydajnościowe, które wykazują, że system osiąga wyniki porównywalne z systemami produkcyjnymi, popularnymi w branży informatycznej, a jednocześnie osiąga założone cele.

Contents

Chapter 1

Introduction

1.1 Distributed systems and the problem with data consistency

When collecting the data in the application, the volume can grow infinitely. Computers, however, have limited amount of resources due to hardware limits or law of physics. In consequence, developers can decide they need scale their application to more than one computer. As the amount of data grows and the amount of computer computers increases, so does the complexity of the system.
Some of the data might be reused more often then other. Querying the data in correct database and then fetching it can be slow,so developers often use specialized software to accelerate this operations, most often in the form of the local cache - a key-value store. However, the data can scale to such enourmous size, that it can no longer be stored locally.
In that case, the distributed cache is used, to balance the load between multiple servers. However the data in cache is meant to be accessed and modified frequently, the conflicts between two server might arise. In most applications, this won't be a problem, because developer can simply refetch the data from database or ignore these uncommon events. However, there are systems that require the data to be correct at all times. This challenge is addressed by the problem of data consistency, which describes how the conflicts are managed and how often they occur.

1.2 Objective and scope of the thesis

The objective of this thesis is to design and implement a distributed in-memory cache, a key-value store, that ensures strong consistency of stored data. Strong consistency implies, that at all times, system behaves as its a single-node entity where every node agrees on the current state. The strong consistency concept will be defined in more depth later in chapter **??**.

It is important to note, that due to the system being a cache rather than a database, data peristence after crash is not guaranteed. Since data is stored exclusively in-memory on the Data Nodes to maximize performance, it remains volatile. If a server and all replicas on which the data is stored crashes or its host servers restarts, the data is lost and has to be refetched from persistent storage such as a database and put again into the system. The primary guarantee of this system is not data persistance after failures, but the consistency - correctness. Whenever user asks for the data and receives a response, this response will be correct and consistent across every server in the cluster. The user interface is implemented as a CLI tool that supports all key-value store operations. The system runs on Docker containers and allows user to easily start itself on a local machine. With some configuration, the user can also start it on many computers and add new servers when needed.

1.3 Technology stack choice

The Go programming language was chosen due to its advanced concurrency model. System relies on the concurrent operations both inside the node and also across the nodes in the cluster. Due to the need for a tool that supports such tasks, the Go was chosen. Furthermore, the system also uses a lot of network intensive operations, which Go excels at.
There were other considerations for languages of the project, namely C++ and Java, but both were rejected. C++ because of its high complexity and the potential impact on the development time considering author's proficiency level. Java was excluded, because it is generally slower than Go in network-intensive tasks and has a higher resource overhead, which we are trying to minimise.
For internal communication I utilized gRPC (a high-performance framework for calling functions on other servers). The choice was made because of gRPC's high performance and efficient binary serialization. For external communication the HTTP protocol was selected due to its simplicity and ease of integration.

1.4 Structure of this thesis

Chapter 2 presents theoretical background necessary to understand the problem of state management in distributed systems. It discusses the CAP theorem, data consistency models, consensus algorithms - what are they, why we need them. It compares two of the most popular consensus algorithms and describe what the chosen one is doing. The chapter reviews existing architectal patterns, sharding techniques, client types and hashing techniques.
Chapter 3 outlines more detailed requirements of the project and the system design. It defines high-level architecture, separation of Control Plane from Data plane. It also details communication between servers and the specific design of client.

Chapter 4 presents a detailed implementation of Raft consensus algorithm. It describes implementation of basic parts of the algorithm aswell as more advanced.
Chapter 5 covers the implementation details of the remaining system components. It describes the logic of the Control Plane, the Data Nodes and the client library. Special attention is given to mechanisms of shard rebalancing and failure detection.
Chapter 6 is dedidacted to testing and verification of the system. It describes the testing methology, including unit tests, end to end tests, integration tests and failure injection tests used to validate system correctness and stability.
Chapter 7 presents the performance analysis and benchmarks in comparison to other industry-standard systems. It contains the results of benchmark tests measuring latency and throughput (requests per second) in comparison to other industry standard systems.
Chapter 8 summarizes the thesis, evaluating the degree to which the project goals were met, discussing the encountered challenges and potential directions for future development.

Chapter 2

Theoretical Foundations and Overview of Solutions

2.1 Terminology

In this thesis, several technical terms are used, some of them interchangeably, to describe valid components of the system. Below are the definitions of each of them.

- **Node / Server / Instance** - refers to a single running process of the application, usually isolated in its own container or physical machine.
- **Cache / Key-Value Store / System** - refers to the entire distributed system designed and implemented in this project.
- **Cluster** - refers to the collective group of all nodes working together.
- **Control Plane / Data Plane** - the system is split into a management layer (Control Plane) and a storage layer (Data Plane).
- **Primary / Replica** - in a shard, the Primary node handles writes and replicates them to one or more Backup (Replica) nodes.
- **Quorum / Majority** - refers to more than half of the total nodes in a cluster (e.g., 3 out of 5).
- **Epoch** - a version number for the cluster topology; it increases whenever the system configuration changes.
- **RPC (Remote Procedure Call)** - a way for one server to call a function or send a message to another server over the network.
- **State Machine** - the component of the server that actually stores the data (like a table of keys and values) and applies changes to it.

- **Synchronous Replication** - a method where the system waits for all replicas to save the data before telling the user it succeeded. This is slow but safe.
- **Asynchronous Replication** - a method where the system tells the user "OK" immediately after the primary saves data, without waiting for replicas. This is very fast but can lose data if someone crashes.
- **Goroutine** - a lightweight, independent task managed by the Go programming language that allows the system to do many things at the same time.
- **Channel** - a "pipe" used by goroutines to communicate and send messages to each other safely.
- **Mutex (Lock)** - a mechanism that ensures only one part of the program can access or change a piece of data at a time to prevent errors.
- **Deadlock** - a bug where two or more parts of a program are waiting for each other to finish, causing the entire system to freeze.
- **Reaper** - a background process that monitors nodes and "reaps" (removes or handles) those that have failed.
- **Throughput** - the total number of operations a system can perform in a given time (e.g., 1000 requests per second).
- **Latency** - the time it takes for a single request to travel from the client to the server and back.

2.2 Consistency models in distributed systems

Distributed systems operate on data. Consistency is a metric that describes how many conflicts between nodes are resolved and how long it takes to resolve them. In this thesis, I will focus on two main categories of consistency models: strong and weak.

2.2.1 Strong Consistency (Linearizability)

This is a strongly consistent model that ensures that all nodes agree on the order in which operations were done. Reads will always return the most recent version of data. When an update occurs, this model makes sure that every other server reflects that change. Due to these guarantees, this model limits how fast the system can work, but allows the system to be seen as a single-node database.

Linearizability is the strongest guarantee for consistency for single-object requests (operations like getting or setting a single key). It gives an illusion that the system is in fact a single copy of data and the data is not distributed. It guarantees

that once a write operation is completed successfully, all subsequent read operations will return that value, or a newer value if any update happened.

Linearizability (the formal definition of Strong Consistency) says:

- If operation A completes (client gets OK), then every operation B that starts after A finishes must see the effect of A.
- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," as long as the system is consistent about that choice from then on.

Mathematically, when operation $Write(x)$ completes at time t_1, then for any given operation $Read(x)$ starting at time $t_2 > t_1$ must see the effects of $Write(x)$.

2.2.2 Eventual Consistency

This is one of weak consistency models that prioritizes availability and performance. It allows data between servers to conflict, but if no updates occur, eventually these conflicts will resolve. In eventually consistent systems, if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value. This model allows for temporary inconsistencies where different nodes serve different versions of data. This approach enables high availability and low latency because writes can be accepted by a single node without waiting for other nodes to accept and make a consensus. This approach, however, requires adding logic for conflict resolution strategies (rules to decide which data is correct, for example, "the newest write wins") when merging data from different nodes. One example can be a like system on a social media platform. If a user likes a friend post, he might see it instantly, but might also see the update after 5 seconds. The operation is not crucial and can have temporary inconsistent data that will resolve after some time, for example when the server synchronizes with others.

2.3 The CAP theorem and trade-offs

CAP theorem, formulated by Eric Brewer in [1], says that a networked system that shares-data, can strictly provide only two of three properties:

- **(Strong) Consistency (C):** Reads will receive the most recent write or an error. If a system is consistent, then all nodes will see the same data at the same time, just like a single-node database would.
- **Availability (A):** Request will always receive a non-error response, without the guarantee that it can contain conflicting data. Availability says that the system remains operational for reads and updates, even if some nodes fail.

- **Partition Tolerance (P):** The system will work even if a network failure occurs or some nodes crash or there are very big delays. The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

In real-world distributed systems, we cannot have Consistent and Available system, because the distributed system itself is a partitioned system, so for it to work it has to be partition tolerant. For example if a hardware failure or network failure occurs, then there is inevitable Network Partition. Hence in real-world distributed systems, the choice is between Consistency and Availability. Most cache systems focus on Availability, because authors wants to maximize performance. Most cache operate on argument, that if a user wants consistency, they should use database. However, there are cases, where the need of Consistency is more important then availability, for example in banking applications or in any system where data correctness is critical.

The distributed systems are either:

1. **CP (Consistency + Partition Tolerance):** When partition occurs, system will shut down all non-consistent nodes or refuse writes if the system cannot support them any longer. This approach prioritizes safety over throughput or uptime.

2. **AP (Availability + Partition Tolerance):** When a partition occurs, system will continue to accept write and read requests, with probability that it will return conflicting data. This approach prioritizes throughput, uptime and user-experience over corectness of the data.

In this thesis, I have chosen to implement a CP system, prioritizing data integrity as the primary requirement for the distributed cache.

2.4 Split brain problem

Split brain is a problem in distributed system where a network divides the cluster into two or more disconnected groups (sub-clusters). Due to disconnection, the nodes cannot communicate with each other, so each node may incorrectly deduct that other nodes in the other group have died. This creates a situation where separate parts of the system operate independently and accept conflicting writes, believing they are the only active group. There are many strategies to manage this problem. Algorithms like Raft only allow a node to become a leader if more than half of total nodes agrees. Fencing is other solution to this problem that makes node physically or logically disable its peer to ensure it cannot write to shared storage.

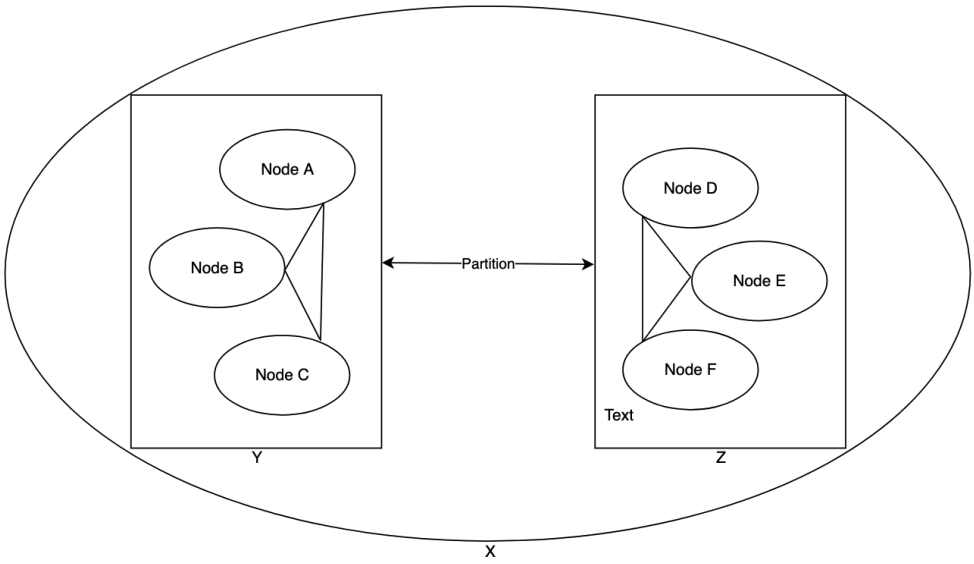

Figure 2.1: Visualization of split brain problem. The network X has been split into subnetworks Y and Z

2.5 Consensus algorithms

A consensus algorithm is a set of rules that allows all servers in a system to reach a single agreement on the same data or sequence of actions. This ensures that every server has exactly the same information at any given time, even if some of them crash or have network problems. It serves as the backbone of the system I developed for this thesis.

2.5.1 Paxos vs. Raft – Comparison of understanding and implementation

Paxos is a family of protocols created by Leslie Lamport [14] and historically the most used algorithm for distributed consensus. While it is mathematically proven to be safe and efficient, it is also a very complex to understand and difficult to implement in real-world systems.

Raft is a modern algorithm, introduced in 2014 by Diego Ongaro and John Ousterhout [2], that was designed to be easier to implement than Paxos and more understandable than it. It offers performance comparable to Multi-Paxos [15] (a more efficient version of the original Paxos). It decomposes the problem of consensus into smaller, independent sub-problems of choosing a coordinator, replicating the sequence of updates and ensuring safety. Raft is widely used in many modern distributed systems such as CockroachDB [3], MongoDB [4], Neo4J [5], RabbitMQ [6].

Key Differences between Paxos and Raft:

- **Central Coordination:** Raft uses a model where one server is responsible for coordinating others. The sequence of updates is always sent from this managing server to other servers. This makes it easier to manage the history of operations compared to Paxos, where data can be sent in more complex patterns between all servers.
- **Continuity of Updates:** Raft guarantees that if a server has an update at position N, it also has all updates from 1 to $N - 1$ which are identical to the coordinating server. Paxos allows for gaps in this sequence that must be filled later, which makes the implementation more difficult.

Due to its focus on implementability and the availability of robust documentation, I have selected the Raft algorithm as the foundation for this project.

2.6 Detailed analysis of the Raft algorithm

Raft ensures that a cluster of N servers can tolerate F failures, where $N = 2F + 1$.

2.6.1 Core concepts and terminologies

Before describing how the algorithm works, it is important to note, that it uses several key concepts. In consequence, understanding these terms is necessary to follow the detailed analysis. For more details, reader should refer to the original Raft paper [2].

- **Leader** - Only one server can be a leader at any time. It manages all client requests and coordinates rest of cluster.
- **Elections** - Since system needs a leader to function, there must be a way to pick one. If current leader stops working, nodes start an election to choose a new one.
- **Logs** - A log is a sequence of entries that store operations sent by users. We need them to keep track of what happened in the system so it can be recovered later.
- **Term** - This is a logical clock. It acts as a version number for the system, helping nodes to identify which information is current and which is old.
- **Safety** - Safety is a set of rules that guarantee that nodes never agree on different things for the same operation. It also guarantees that node who crashed and rebooted will not become a leader until the data it has is up to date.

Leader Election

Raft uses a heartbeat mechanism to maintain leadership. If no heartbeat is received in some configurable time window, then server can assume the leader is dead and start a new election.
The election start by the node incrementing its current `Term` (a logical clock used to detect stale information), changing its current state to the Candidate state, voting for itself, and broadcasting `RequestVote` RPCs to all other nodes. A candidate wins if it receives votes from a majority of the cluster. We want to have randomized timeout for election start because we want to minimize the probability of multiple nodes starting election at the same and because of that not one receiving a majority, which would cause system stalls.

Log Replication

Once elected, the leader accepts client commands. A command flows through the following lifecycle:

1. The leader appends the command to its local logs.
2. It sends `AppendEntries` RPCs to all followers.
3. Once a majority of followers acknowledge the entry, the leader marks the entry as **committed**.
4. The leader applies the entry to its state machine and returns the result to the client.
5. In subsequent heartbeats, the leader notifies followers of the new `commitIndex`, instructing them to apply the data to their own state machines.

Safety

Raft ensures safety via the **Election Restriction**. Candidate cannot win election if its log is not in the most up to date state. During the voting process, a voter denies its vote if the candidate's log is less up to date than its own by comparing the index and term of the last log entry. Additionally nodes utilize a **Term** number to detect and reject stale leaders. If a node receives a request from a leader from a leader with a lower term than its own, then it will reject that request and force a stale leader to step down. This mechanism guarantees the State Machine Safety property, that is, if a server has applied a log entry at given index to its state machine, then no other server will ever apply a different entry at the same index. This guarantees the prefix is also the same.

2.7 Cache system architectures

Consensus algorithm ensure the coordination of servers is correct, but architectural design is what allows the system to scale to handle gigabytes of data and thousands of requests per second.

2.7.1 Sharding and Consistent Hashing

To distribute the data between multiple server, the process called sharding is introduced. Sharding is a process of partitioning data into multiple segments, each stored on a different server.

Due to the need of deciding which server should store which piece of data, we use a technique called hashing. A hash is a function that takes a key and turns it into a specific number. If we calculate a hash for a key and the result points to server X, then the data for this key is placed there. In consequence, the system can quickly know where the data is, without asking every server in the cluster.

Traditional Modulo Hashing

The most basic approach would just use $server = \text{hash}(key) \pmod N$, where N is the number of servers. This method is fine in a static environments where the number of servers remains constant. However if we need to scale dynamically, changing N causes the result of modulo operation to change nearly all keys in the system. This is highly ineffective.

Consistent Hashing

Consistent hashing maps both data and servers onto a logical ring topology (0 to $2^{32} - 1$). A key is assigned to the first server encountered moving clockwise on the ring. [9]

When a server is added or removed, only K/N keys need to be remapped (where K is the total keys), which significantly minimizizes data movement.
When some part of the ring is used way more often then other we use **Virtual Nodes**. Each physical server is placed onto the ring multiple times (e.g., at 100 different positions). This statistical distribution ensures a more uniform load balancing across the cluster.

2.7.2 Role of the client: Thin Client vs. Smart Client

A crucial decision is on how to client interacts with the system:

- **Thin Client (Proxy-based):** The client is unaware of the cluster topology. It sends requests to a load balancer or a random node (acting as a proxy), which then forwards the request to the correct shard.
  - *Pros:* Simple client implementation; topology changes are transparent to the application.
  - *Cons:* Additional network hop (Client → Proxy → Shard), increasing latency.
- **Smart Client:** The client maintains a local map of the cluster topology (slots mapping to node IP addresses). It computes the hash and connects directly to the target node.
  - *Pros:* Lowest possible latency (single hop); no bottleneck at a central proxy.
  - *Cons:* Complex client logic; the client must listen for cluster configuration changes (e.g., failovers or resharding) to update its internal routing table.

In this thesis, I have implemented a Smart Client to improve the latency time and for scalability.

2.8 Overview of existing solutions (Redis Cluster, Etcd) - Analysis in terms of consistency

2.8.1 Redis Cluster

Redis Cluster is the industry standard for distributed caching. It employs a multi-master architecture with asynchronous replication.

- **Architecture:** It uses sharding with 16,384 hash slots. Each master node owns a subset of slots and has one or more replicas.
- **Consistency:** Redis prioritizes Availability and Performance (AP-leaning). By default, replication to slaves is asynchronous. If a master crashes after accepting a write but before replicating it to a slave, and that slave is promoted to master, the write is lost. Redis does not guarantee strong consistency; it provides eventual consistency and high throughput.

2.8.2 Etcd

Etcd is a distributed key-value store used primarily for shared configuration and service discovery (e.g., in Kubernetes).

- **Architecture:** It is built strictly on the Raft consensus algorithm. Unlike Redis, it does not shard data by default; every node replicates the full state machine.
- **Consistency:** Etcd is a CP system guaranteeing strict serializability. Every write goes through the Raft leader and must be committed to a quorum before returning success. This ensures data safety at the cost of lower write throughput compared to Redis.
- **Durability:** Etcd is a system that guarantees durability: Even if a node crash, the data is not lost.

The solution implemented in this work aligns more closely with the Etcd model regarding consistency (using Raft for strong guarantees) but incorporates architectural elements of Redis (key-value storage focus) to explore the performance implications of strong consistency in a caching scenario. My solution **does not** guarantee durability, focusing instead on in-memory performance while maintaining strong consistency.

2.8.3 Analysis of hashing and partition approaches

Karger's Consistent Hashing

Proposed by Karger et al. at MIT in 1997 [9], consistent hashing solves the reshuffling problem inherent in modular hashing. In a standard modulo setup ($\text{hash}(k) \pmod n$), changing n (adding/removing a node) results in nearly all keys being remapped. Karger's approach maps both the **Nodes** and the **Keys** onto the same identifier space (a ring).

Why it was not used: While industrially popular (consistent hashing is used in Cassandra [13] and Dynamo [10]), it introduces complexity in the Controller's metadata management. Maintaining a sorted ring and handling "virtual nodes" requires more complex logic than a fixed partition map. For this thesis, I opted for a **Fixed Slot** approach (similar to Redis) because it allows for deterministic shard-to-node mappings that are easier to synchronize via Raft.

Range Partitioning (Sorted Monolithic Map)

This approach treats the entire key-space as a single, sorted monolithic map. The data is divided into contiguous ranges (e.g., keys $[A, K)$ on Node 1, $[K, Z)$ on Node 2).

- **Pros:** Allows for efficient range queries (e.g., `SCAN user:100:*`).

- **Cons:** Suffers from the **Sequential Write problem**. If keys are generated with a monotonically increasing prefix (like timestamps), all traffic targets a single node, creating a hot spot.

Why it was not used: A cache's primary goal is often uniform load distribution for point Lookups (GET/PUT). Range partitioning is better suited for analytical databases (like CockroachDB [3]). Given the use of Raft for the Control Plane, a hash-based distribution ensures even load across Data Nodes without the need for complex range-splitting logic.

FNV-1a Hashing Algorithm

The Fowler-Noll-Vo (FNV) hash 1a [12] is a non-cryptographic hash function known for its extreme speed and ease of implementation. It works by multiplying the hash by a large prime and XORing it with each byte of the input data.

- **Pros:** High throughput, low collision rates for short strings (ideal for cache keys), and minimal memory overhead.

- **Suitability:** Since this project requires mapping keys to a fixed set of shards during every client request, a low-latency hash function like FNV-1a is preferable over more complex cryptographic hashes like SHA-256 or MD5, which would introduce unnecessary CPU overhead.

2.8.4 Analysis of industry inspirations

Redis Cluster: The "Fixed Shards" Inspiration

Redis Cluster pre-shards the key-space into 16,384 "hash slots". A key is mapped to a slot using a mathematical formula called `CRC16`: slot = CRC16(key) (mod 16384) (where modulo gives the remainder of the division).

- **Inspiration:** My implementation adopts this "Fixed Shard" model. By mapping keys to a constant number of shards (e.g., 10 or 1024) the system decouples the data distribution from the number of physical nodes.

- **The Difference:** Redis uses asynchronous replication by default to maximize performance, which can lead to data loss. This system uses **Synchronous Replication** to ensure Strong Consistency, sacrificing a degree of write latency for the safety guarantees required by the thesis.

Amazon Dynamo: Availability vs. Consistency

Dynamo is the archetypal "AP" (Available, Partition-Tolerant) system. It uses consistent hashing and eventual consistency mechanisms like **Vector Clocks** (to track versions of data) and a **Gossip protocol** (where servers share updates with each other like neighbors sharing rumors) to ensure it is "always on" [10].

- **Inspiration:** Dynamo's concept of a "Preference List" for replication was an inspiration for how the Controller manages shard assignments.
- **The Difference:** Dynamo's philosophy is "eventual consistency." It allows conflicting writes and resolves them later. This system purposefully takes the opposite path: identifying as a "CP" system that uses Raft and fencing to prevent conflicts from ever occurring, matching the strict requirements of linearizability.

TiKV: Raft-per-Shard vs. Centralized Control

TiKV is a modern distributed KV store where every "Region" (shard) is its own independent Raft group [11].

- **Comparison:** TiKV's approach is the gold standard for massive scalability. However, running 1000 shards would mean running 1000 independent Raft state machines per node, which is computationally expensive.
- **My Approach:** To balance performance with safety, I separated the system into a **CP Control Plane** (Raft) and a **Synchronous Data Plane**. This significantly reduces the overhead; only the cluster metadata (topology) requires the full weight of the Raft algorithm, while data replication is handled by a simpler, faster synchronous **Primary-Backup protocol** (where the primary just forwards the data directly to its replicas).

Chapter 3

Requirements Analysis and Software Architecture Design

3.1 Functional and Non-functional Requirements

Functional requirements define specific behaviors and functions the system must provide. In the context of this thesis, I have established the following functional requirements:

- Support GET (reading), PUT (saving), and DELETE (removing) operations via an HTTP API (a standard way for different software to talk to each other over a network).
- Automatic sharding of the keyspace across a configurable number of shards.
- Automatic replication of each shard (primary + replicas).
- Dynamic cluster membership - data nodes can be added and removed at runtime.
- Rebalancing when nodes join, leave, or fail.
- A Smart Client that routes requests to the correct node and caches the topology.

Non-functional requirements define how the system operates and what it guarantees in terms of performance, security, usability, and reliability:

- Strong Consistency - the system must reject requests when a quorum cannot be reached; the system guarantees that all successful responses are correct.
- Partition Tolerance - the system continues to operate even if some of the nodes fail or are disconnected from others.

- Fault Tolerance - the system detects node failures and triggers automatic failover - the process where a healthy server takes over the work of a failed one.
- High Availability during maintenance - no downtime when adding or removing nodes and redistributing shards between data nodes.

3.2 High-Level Architecture

The system is split into two logical parts: the Control Plane and the Data Plane.

3.2.1 Separation of the Control Plane and the Data Plane

The Control Plane is responsible for:

- Running a Raft consensus algorithm that stores cluster metadata: topology, shard mapping, node status, and an epoch counter.
- Exposing internal HTTP endpoints used by DataNodes for pull-based shard migration.
- Performing node failure detection (via the ‘Reaper‘ - a background process that checks if nodes are still alive) and initiating rebalancing.

The Data Plane is responsible for:

- Storing the actual key-value data.
- Handling client RPCs (GET, PUT, DELETE).
- Reporting heartbeats (regular ”I am alive” signals) to the Control Plane.

3.2.2 gRPC Communication Protocol and Interface Definition

gRPC is a high-performance Remote Procedure Call (RPC) framework developed by Google. It uses Protocol Buffers as its interface definition language (a set of rules for how messages should look) and message serialization format (the process of turning data into a compact binary format that can be sent over the wire). This provides an alternative to traditional JSON-over-HTTP, which results in smaller payloads (the actual data being sent) and faster processing.

The system utilizes a dedicated gRPC service for the Control Plane’s Raft operations. We define six core methods and ten data types in the `raft.proto` file:

Table 3.1: gRPC Service Methods for Raft Consensus

Method	**Responder**	**Purpose**
`Forward`	Leader	Forwards a client write request to the Raft leader.
`ForwardGet`	Leader	Forwards a client read request to the Raft leader.
`LogRequest`	Follower	Standard Raft *AppendEntries* for log replication.
`VoteRequest`	All	Standard Raft *RequestVote* for leader election.
`Heartbeat`	All	Basic reachability check used in health monitoring.
`InstallSnapshot`	Follower	Transfers chunked state machine data to lagging nodes.

These functions form the backbone of the Control Plane's consistency. Detailed implementation of how these RPCs interact with the Raft state machine is discussed in Chapter 4.

3.3 Design of the Control Plane Module

3.3.1 Cluster Topology and Metadata Management

Topology data includes:

- Nodes map: node ID mapped to NodeMetadata (address, status, last heartbeat timestamps).
- Shards map: shard ID mapped to ShardMetadata (primary ID, replica ID, status).
- Epoch: a monotonically increasing identifier used for conflict-free updates.

The Controller updates the cluster topology using the Raft algorithm. I want to ensure that the Control Plane provides strong consistency, because if it did not, then the system as a whole would not be strongly consistent. This is because, if there were conflicts in the Control Plane, a client might send a request to the wrong DataNode, hence creating an inconsistent state which would fail to guarantee a "correct response on every request".

Rebalancing uses the Control Plane's active nodes to determine which node should be primary and which should be a replica. It then writes the new configuration to the Raft Log, which is distributed among every other Controller in the Control Plane.

3.3.2 Node Failure Detection Mechanism

I implemented a Reaper mechanism which tracks the last heartbeat timestamp of each node. In the background, the Reaper runs a thread that checks if a node has sent a heartbeat within the configured grace period. If it hasn't, the Reaper invokes a callback to the Control Plane, triggering a topology rebalance.

3.4 Smart Client Design

3.4.1 Query Routing Strategy and Topology Caching

Upon initialization, the client connects to the Control Plane to fetch the topology, which it stores in a local cache. For each request, the client hashes the key to a shard ID and looks up the primary node from the cached topology. The request is then sent. On RPC failure, the client retries the request against the current primary. If the primary is unreachable or the request is rejected, the client refetches the cluster topology (as described in Section 3.4.1).

Chapter 4

Implementation of the Raft Consensus Module (pkg/raft)

The core of the distributed system is the Raft consensus algorithm, located in the `pkg/raft` package. This module is responsible for managing the state of distributed nodes, ensuring data consistency across the cluster, and handling leader election and failure recovery. While the Data Plane uses synchronous replication for performance, the Control Plane requires the formal correctness and safety guarantees of Raft to manage the cluster's critical metadata.

The implementation is highly modular, decomposing (breaking down) the Raft protocol into several distinct components: `Elector`, `Replicator`, `DataSaver`, and `Snapshotter`. This separation of concerns allows for a clean concurrency model (a way to manage multiple tasks at once) where each component operates on its own set of goroutines, communicating via channels to minimize lock contention (a situation where many tasks wait for the same lock, slowing down the system).

4.1 Raft Node State Machine

The system defines the node's lifecycle through three primary roles: `Follower`, `Candidate`, and `Leader`. The transition between these roles is strictly controlled by the term logic and majority voting rules.

4.1.1 Implementation of Roles: Follower, Candidate, and Leader

The roles are defined as typed constants in `types.go`: `Follower`, `Candidate`, and `Leader`.

- **Follower:** The default state upon initialization (in `NewRaft`). Followers passively respond to RPCs from leaders and candidates.

- **Candidate:** A node transitions to this state when an election timeout occurs. This logic is encapsulated within the `startElection` method in `election.go`.

- **Leader:** The node that handles all client requests and coordinates log replication. Transition to this role is handled by the `becomeLeader` function in `utils.go`, which initializes the `Replicator` instances for all peers.

4.1.2 Concurrency and Locking Model

Unlike simple implementations that wrap the entire state in a single mutex, this module uses a `sync.RWMutex` (a special lock that allows many readers but only one writer) for the core Raft state, while sub-components often use their own internal synchronization. A critical design choice was the delivery of committed logs to the `Application` interface; to prevent deadlocks and blocking the consensus loop, the mutex is explicitly unlocked during the `deliverToApplication` phase, allowing the application to process data without stalling incoming RPCs.

4.1.3 Leader Election Mechanism (election.go)

The `Elector` manages the node's election timer using a randomized timeout strategy (400–800 ms) to reduce the probability of split votes in large clusters.

1. **Timer Loop:** A dedicated goroutine runs `electionTimerLoop`, which listens on a `resetTimerCh`. This channel allows other components (like heartbeats or valid AppendEntries RPCs) to reset the timer efficiently.

2. **Safety Checks:** Before starting an election, the node verifies it is not currently the leader and is not in the middle of installing a snapshot, as snapshot installation is a heavy I/O operation that might temporarily delay heartbeat processing.

3. **Parallel Requests:** When a node becomes a Candidate, it broadcasts `VoteRequest` RPCs to all peers in parallel using individual goroutines, ensuring the election process is as fast as the fastest majority of the network.

4.2 Log Replication Logic (replicator.go)

The replication logic is implemented using an isolated "per-peer" model. For every follower in the cluster, the leader spawns a dedicated `Replicator` goroutine.

4.2.1 Isolated Replicators and the Pulse Protocol

Each `Replicator` maintains its own state for a specific peer, including `nextIndex` and `matchIndex`. This provides several advantages:

- **Fault Isolation:** A slow or disconnected follower does not block replication to other healthy nodes.
- **Instant vs. Periodic Replication:** The replicator waits on a `signalCh`. When the leader receives a new client request, it "pulses" this channel to trigger immediate replication. If no writes occur, a 50 ms ticker ensures periodic heartbeats are sent to maintain leadership.
- **Self-Healing Quorum:** If a replicator detects too many consecutive failures (defined in `maxConsecutiveFailures`), it triggers a `checkQuorumHealth()` call, which can cause a leader to step down if it realizes it no longer has a healthy majority.

Replication is triggered via the `LogRequest` RPC (standard Raft *AppendEntries*). The `Replicator` runs a continuous loop that monitors a signal channel. When triggered, it calculates the `prevLogIndex` and `prevLogTerm` to ensure log consistency. These arguments are packaged into a `LogRequestArgs` Protocol Buffer message and sent via the `PeerClient` interface.

4.2.2 gRPC Connection Management

Maintaining stable gRPC connections in a dynamic cluster is handled by the `ConnectionManager`.

- **Lazy Initialization:** Connections to peers are only established when needed (e.g., during the first replication attempt or vote request).
- **Exponential Backoff with Jitter:** If a connection fails, the manager uses a backoff strategy (waiting longer and longer between retries, starting at 100 ms up to 5 s) with added randomization (jitter). This prevents the "Thundering Herd" problem where all nodes try to reconnect to a recovering leader at the exact same moment, potentially overwhelming its network capacity.
- **Health Monitoring:** The manager periodically verifies the state of idle connections, closing and reopening them if they become stale or if an underlying network partition is detected.

4.2.3 Committing Entries (Commit Index) and Applying to the State Machine

The leader tracks the `matchIndex` (represented as `ackedLengths`) for each follower. When a majority of nodes acknowledge a log entry, the leader advances its `committedLength`. This logic is implemented in the `commitLogEntries` function in `raft.go`.

Upon commitment, messages are applied to the local state machine via the `Application` interface (defined in `application.go`). The application executes the command (e.g., updating the in-memory cache) and returns the result, which is then sent back to the client via a response channel.

4.2.4 The Log Request (AppendEntries)

The protocol uses a sophisticated `prepareLogRequestArgs` method to calculate the exact suffix of the log that needs to be sent. It handles the transition between sending incremental log entries and triggering a full snapshot installation if the follower's lag exceeds the available logs in memory.

4.3 Asynchronous Persistence Management

To survive crashes and restarts, the Raft module implements strict persistence guarantees using the `DataSaver` struct located in `persistence.go`.

4.3.1 Writing Raft State to Disk (persistence.go)

The persistence layer uses Go's `encoding/gob` package (a specific binary format) to serialize data structures to disk. The `DataSaver` utilizes a worker pool pattern (a group of background tasks ready to handle work) to handle save requests asynchronously, preventing disk I/O (reading and writing to disk) from blocking the main Raft consensus loop.

The state is split into two files:

- **Metadata File:** Stores the `currentTerm`, `votedFor`, and `committedLength`.
- **Logs File:** Stores the append-only sequence of `LogEntry` objects.

4.3.2 State Recovery

Upon restart, the `LoadValues` method performs a sequential scan of the log file, reconstructing the in-memory log slice. Because logs are append-only (new data

is only added to the end), the system handles partial writes by ignoring corrupted entries at the end of the file during recovery.

4.4 Log Compaction and Snapshotting

To prevent the log file from growing infinitely, the system implements a managed snapshotting mechanism.

4.4.1 Snapshot Trigger and Creation

When the log length exceeds the `snapshotThreshold`, the leader initiates a snapshot. It requests a serialized state from the `Application`, writes it to a file, and then truncates (removes the old, already saved entries) the in-memory log.

4.4.2 Chunked Snapshot Transfer (InstallSnapshot)

Transferring a large snapshot over gRPC can be problematic as it might exceed the maximum message size or cause network timeouts. I implemented a `Chunked Transfer Protocol` in `sendInstallSnapshotRPC`:

- The snapshot file is read in 128 KB blocks.
- Each block is sent as a separate `InstallSnapshotRequest` containing an `offset` and a `Done` flag.
- The follower reconstructs the file using `WriteAt`, ensuring that the snapshot is durable on the follower's disk even if the transfer is interrupted and resumed.

This mechanism allows the system to synchronize new nodes even when the dataset is significantly larger than available network buffers.

4.5 System Configuration and Environment Parameters

The Raft module is highly configurable through environment variables, allowing it to be tuned for different network conditions or testing scenarios.

Table 4.1: System Configuration Parameters

Variable	**Default**	**Description**
`RAFT_ID`	*required*	Unique integer ID for the Raft node.
`TOTAL_NODES`	*required*	The total number of nodes in the cluster.
`RAFT_ADDRS`	*required*	Comma-separated list of all node addresses.
`SNAPSHOT_THRESHOLD`	*required*	Number of log entries before a snapshot is triggered.
`RAFT_INITIAL_BACKOFF`	1 s	Initial delay for gRPC reconnection attempts.
`RAFT_MAX_BACKOFF`	30 s	Maximum delay for reconnection attempts.
`RAFT_ARTIFICIAL_DELAY`	0 ms	Simulates network/CPU lag for performance testing.

Chapter 5

Implementation of the Control Plane and the Data Plane

5.1 Implementation of the Control Plane

The Control Plane node is the central orchestrator of the entire application. It is responsible for maintaining the system state and ensuring that all nodes remain synchronized.

5.1.1 Topology State Machine

The core of the Controller is a Replicated State Machine (a system where every controller has its own identical copy of the state) responsible for managing cluster metadata. This ensures that, in a multi-node setup, all Controller instances agree on which DataNode holds which shard.
The state represents:

- Epoch: A monotonically increasing number representing the version of the topology. This works similarly to a Raft term.
- Nodes: A map of all registered DataNodes with information about their addresses and statuses.
- Shards: A map of all shard IDs to their primary and replica DataNode IDs.

The Controller implements the Raft application interface, processing commands such as node registration and topology updates. Changes to the topology are not applied until they are committed by the Raft algorithm to ensure strong consistency. When a command is sent to the Controller, it updates its local configuration and, if the Controller is the leader, it may trigger a rebalance. To prevent the Raft

log from growing infinitely, the system supports snapshots, which save the current cluster configuration to durable storage.

5.1.2 Node Health Monitoring

A dedicated component called the Reaper (as defined in the Terminology section) is responsible for monitoring the system for failures. DataNodes are designed to periodically send heartbeats to the Control Plane, which updates the timestamp of the last received heartbeat for each node. The Reaper runs a background goroutine that wakes up every second to check the liveliness of all DataNodes. If a node has not sent a heartbeat within the configured grace period, it is marked as DEAD. In this case, the Reaper triggers a callback that signals the Controller to take action. If the failed node was a primary, one of its replicas is promoted to the primary role. Subsequently, the rebalancing logic will attempt to find a new node to act as a replica to restore the desired redundancy level.

5.1.3 Shard Rebalancing

Rebalancing is responsible for ensuring that data availability and the even distribution of data are preserved.

When a node failure is detected, the system performs an emergency promotion:

1. The Controller identifies all shards where the failed node was the primary.
2. For each such shard, the first available replica is promoted to become the new primary. Since the original primary is dead, no data can be copied; the promoted replica relies on the data it already possesses.
3. The dead node is removed from the active node collection.

Note: When both the primary node and its replica die, the data in those shards is lost. This is a direct consequence of not providing a durable cache, as noted previously. While it is possible to increase the number of replicas, doing so will slow down the cache, as the system must ensure the primary DataNode replicates data to every replica before returning an OK status to the client. However, it is very unlikely for both nodes to die simultaneously if they are located on different physical servers.

Rebalancing is triggered automatically when a new node registers or a failure occurs. It uses a greedy strategy:

1. It identifies all shards that lack a primary node and assigns them to ACTIVE

nodes using a round-robin approach (a simple method of rotating through available servers one by one).

2. It ensures that every shard has the required number of replicas, selecting new nodes to act as replicas if needed.

3. For initializing these new replicas or during planned load balancing, a synchronization protocol is used to ensure data consistency.

To initialize new replicas, the Controller implements a four-phase synchronization protocol:

1. The target node is instructed to perform a bulk copy of all data from the current primary node.

2. The shard is marked as LOCKED in the cluster topology. This temporarily stops writes to this specific shard to ensure a consistent state.

3. A final incremental catch-up is performed to transfer any data that was added during the initial bulk copy in phase 1.

4. The target DataNode is officially added to the replica set, and the shard is set back to ACTIVE.

5.1.4 The NoOp Command

The `NoOp` command is a special administrative command in the Raft state machine that performs no data modification. Its primary purpose is to refresh the leader's knowledge of the current state and verify that it still holds a valid quorum. In this implementation, the Controller uses `NoOp` to clear any potential uncertainty after an election and to ensure that topology updates are only issued by a leader that is fully synchronized with the cluster.

5.1.5 Controller HTTP API Reference

The Control Plane exposes several internal and external HTTP endpoints. These are used by DataNodes for registration and by the Smart Client for topology discovery.

Table 5.1: Controller HTTP API Endpoints

Endpoint	**Method**	**Description**
`/register`	POST	Registers a new DataNode with its address.
`/heartbeat`	POST	Receives health updates and returns the current Epoch.
`/topology`	GET	Returns the full JSON description (a standard text-based data format) of
`/pull`	POST	Triggers a shard migration from a source to a target node.

While emergency promotion handles sudden failures, the system also manages the lifecycle of nodes entering and leaving the cluster gracefully.

Scaling Up: The Lazy Assignment Strategy

When a new DataNode registers, the `Rebalance()` function is invoked. In the current implementation, this follows a *Lazy Assignment* strategy:

- New nodes are primarily used to fill "gaps" in the cluster—specifically, shards that do not have the required number of replicas or shards that have no assigned primary DataNode.
- To prevent unnecessary data movement (which can saturate the network), the system does not automatically "steal" shards from existing healthy primary nodes.
- For active load balancing (manual redistribution), an administrator can trigger the `MoveShard` command, which utilizes the four-phase synchronization protocol to migrate a primary role to the new node with zero data loss and minimum downtime.

Scaling Down and Replica Recovery

The system treats a deliberate node removal similarly to a failure. If a node is decommissioned:

- **Redundancy Restoration:** If the removed node was a replica, the `Rebalance` logic detects that the shard's replication factor has decreased. It automatically identifies another healthy node in the cluster and triggers a data pull to restore the backup.
- **Safe Promotion:** If the node was a primary, promotion happens first, followed by replica restoration on a different node.

This ensures that the cluster is **self-healing** regarding its durability guarantees; as long as there is spare capacity in the cluster, the system will always attempt to maintain the configured number of replicas for every shard.

5.2 Data Plane

5.2.1 Concurrent Data Store

The data storage system provides a simple but thread-safe in-memory key-value store. It uses a standard Go map structure wrapped with a mutex. The data store

supports data migration and, in particular, implements two core functions:

1. `ExportShard(shardID, totalShards)` : A crucial function for rebalancing, it iterates through the entire map, hashes each key to determine its shard, and returns only those keys belonging to the given shard ID.

2. `Import(data)` : This function bulk-loads data into the map and is used during shard migration or recovery.

5.2.2 Lease Management and Fencing

Lease management is the system's primary defense against the split-brain problem. DataNodes are implemented to send heartbeats to the Control Plane to signal that they are alive. If the Controller acknowledges the heartbeat, the DataNode extends its validity timestamp. A lease is a configurable duration during which a DataNode is considered valid. We use fencing to verify the lease by comparing the current time to the last validity timestamp. If the difference exceeds the lease time, the DataNode immediately stops serving requests. To ensure this condition is always checked, every PUT, GET, and DELETE operation verifies the lease. If the lease has expired, the DataNode returns an HTTP `503 Service Unavailable` response, effectively fencing the node out of the cluster.

5.2.3 Synchronization and Consistency

The Data Plane maintains consistency through coordination with the Control Plane.

The Data Plane ensures that the topology is always synchronized. Heartbeat responses contain the latest Epoch. If the Epoch is higher than the node's local Epoch value, the lease manager will immediately fetch the latest topology from the Control Plane and update the local state. This ensures that the DataNode is always aware of the role assigned to it by the Control Plane—whether it is a primary node or a replica.

The Data Plane also uses epoch-based fencing. If a node has an active lease, every write request must include an Epoch. The DataNode compares the client's Epoch to its own; if the request's Epoch is smaller than the local one, the request is rejected with a `412 Precondition Failed` status, preventing stale writes from delayed clients that have yet to refetch the current topology.

The final synchronization mechanism is primary-backup replication. If a DataNode is a primary, it is responsible for replicating writes to all replica nodes. This

replication is **synchronous** (meaning the system waits for all replicas to finish before returning a success message to the user). This makes the system **strongly consistent**, as a request is only accepted if all replicas and the primary node have successfully saved it. This also provides a form of durability, as the system preserves data as long as at least one replica remains alive.

Note: The strong consistency definition in Chapter **??** states that:

- If operation A completes (the client receives an OK), then every operation B that starts after A finishes must see the effect of A.
- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," provided the system remains consistent about that choice thereafter.

If all replicas receive a write request but the primary node dies before acknowledging it, we encounter a state of uncertainty—the second scenario. However, since all replicas recorded the update, the client will see the request as accepted once the failover completes. In the case where only some replicas received the write before the primary crashed, the system might be inconsistent. This is resolved by the Smart Client via client-side idempotency (a mechanism that ensures an operation can be safely repeated without changing the result), which is discussed in more detail in Section 5.2.8.

5.2.4 Data Transfer Protocol

Shard synchronization is a crucial part of the system. Two DataNodes—a source and a target—must synchronize their data. The source node exports its data, while the target node imports it. The Data Plane uses a pull model:

1. The Controller sends a pull request to the target node.
2. The target node sends a GET request to the source node's export endpoint, which returns the result of the `ExportShard()` function (see Section 5.2.1).
3. The target node calls the `Import()` function (see Section 5.2.1) using the data received from the source node.

5.2.5 DataNode HTTP API Reference

Data Plane nodes expose endpoints for client CRUD operations and internal synchronization.

Table 5.2: DataNode HTTP API Endpoints

Endpoint	Method	Description
`/data`	GET	Retrieves a value by key if the node is the primary.
`/data`	POST/PUT	Stores a key-value pair and replicates to backups.
`/data`	DELETE	Removes a key and replicates the deletion.
`/replicate`	POST	Internal: Replica receives a data write from the primary.
`/delete-replicate`	POST	Internal: Replica receives a delete command from the primary
`/export`	GET	Internal: Exports shard data for migration.
`/import`	POST	Internal: Imports shard data from a source node.

Shard Locking: During migration (Phase 2 of the four-phase protocol), the shard is set to `StatusLocked`. Any client request to a locked shard is rejected with an HTTP `423 Locked` status, signaling the Smart Client to wait and retry.

While the Control Node handles high-level cluster orchestration and the DataNode handles lower-level data logic, the Client and Access Library are responsible for efficient operations and ensuring reliable communication.

5.2.6 Smart Client Mechanism

The Smart Client is an implementation that eliminates the need for a centrally implemented load balancer or proxy, which can increase latency or compromise data consistency (discussed in more detail in Chapter 3).

The client maintains a local copy of the cluster topology fetched from the Control Plane. At the start of a session, the client queries one of the Controllers to obtain the latest topology. It must hold its own Epoch to include in every request and verify whether its topology state is current. The client also tracks the list of active DataNodes and shard-to-node mappings to route requests correctly based on the key's hash.

Every GET, PUT, and DELETE operation is associated with a key. The client calculates the hash of the key to route the request to the correct node:

1. Hash the key using the FNV-1a algorithm (see Section 2.8.3).
2. Determine the shard ID by taking the hash modulo the number of shards.
3. Look up the primary DataNode ID for that shard in the local cache.
4. Send the request to the identified primary DataNode.

The client is lazy but reactive, continuing to use its local cache as long as it remains valid. Cache validity is determined by DataNode responses:

1. `412 Precondition Failed`: The client's Epoch is too old; the DataNode has a newer Epoch.
2. `400 Not Primary`: The shard has migrated or a failover has occurred.

In both cases, the client must refetch the topology and associated metadata from the Control Plane (as described in Section 3.4.1) and update its local cache.

5.2.7 Adaptive Retry and Backoff Strategy

To ensure resilience against transient failures, the Smart Client implements an adaptive retry loop:

- **Retry Limit:** Each operation is attempted up to `maxRetries` (default: 5) times.
- **Role Miss Retry:** If a node returns `400 Not Primary`, the client immediately refetches the topology and retries the request on the new primary.
- **Migration Wait:** If a shard is locked (`423 Locked`), the client waits for 100 ms before retrying, allowing the migration protocol to complete Phase 2.
- **Failover Wait:** If a node is fenced or unreachable (`503 Service Unavailable`), the client waits for 200 ms to allow the Control Plane's Reaper to detect the failure and trigger an emergency promotion.

This tiered wait strategy balances system responsiveness with the need to avoid overwhelming a cluster that is undergoing reconfiguration.

5.2.8 Client-Side Idempotency

To prevent duplicate operations during retries (especially when a primary node has died), the client includes an idempotency token and a client ID in each request. Every write and delete operation uses a unique UUID token, which is used to track the status of the operation. Each log entry in the Raft algorithm is designed to hold the idempotency token and client ID. If the client is unsure whether an operation succeeded, they can safely resend the request; if it has already succeeded, the system will identify the previous request and ensure no duplicate processing occurs. If it failed, the system will retry it.

This mechanism ensures that the primary node in the Data Plane acts as the coordinator for the token. When writes are replicated, the token is passed along to the replicas. If the primary node fails after replicating but before acknowledging the client, the promoted backup already possesses the request and its token. If the

client then resends the request, the new primary will recognize the token and reply with an OK status. This solves the previously discussed problem where only some replicas receive a write before a primary crash (see Section 5.2.3).

Imagine a situation with a primary node P and two replicas R_1 and R_2. The client sends a request `PUT(key, value)` with token T. Primary node P replicates to R_1, which saves the request, but P crashes before replicating to R_2 or acknowledging the client. The client request times out, and the client resends it. We have two possibilities:

1. R_1 becomes the new primary: It recognizes the request with token T and returns OK. Through shard synchronization (see Section 5.2.4), R_2 will eventually receive this update, maintaining a consistent state.

2. R_2 becomes the new primary: Since R_2 did not receive the initial request, it will process the resent request, write it to memory, and replicate it to R_1 (overwriting any partial state). The system remains consistent.

This logic extends naturally to any number of replicas.

5.3 Control Plane API Layer (pkg/api)

The `pkg/api` package provides the external interface for the Control Plane. While internal cluster coordination uses gRPC, administrative tasks and Control Plane state queries are exposed via a traditional HTTP/JSON API.

5.3.1 Bridging Raft and HTTP

The `api.Server` acts as a wrapper around the Raft node. When an HTTP request arrives (e.g., to update a configuration), the server must ensure the request is handled by the current leader.

- **Leader Caching:** To avoid the latency of checking leadership status on every request, the API server maintains a `LeaderCache`. It caches the address of the known leader for a short period (1 s), refreshing it only if a request to that leader fails.

- **Command Forwarding:** If a client hits a follower node with a write request, the follower uses the gRPC `Forward` method to send the command to the leader. The leader then proposes the command to the Raft log.

5.3.2 Idempotency and Resilience

Distributed consistency requires handling retries safely. The API layer implements two critical sub-components:

1. **Idempotency Cache:** To prevent the same administrative command from being executed twice (for example, registering the same DataNode multiple times due to a network timeout), the server tracks request IDs in an `IdempotencyCache`. It discards duplicate requests before they reach the Raft log.
2. **Auto-Retrier:** Transient failures are handled by a `Retrier` component, which automatically retries idempotent operations if the connection to the leader is interrupted, ensuring high availability for Control Plane operations.

5.3.3 Control Plane Client API Reference

The following table lists the endpoints exposed by the API server for external interaction:

Table 5.3: Control Plane Client HTTP API

Endpoint	**Method**	**Description**
`/kv`	POST	Proposes a generic KV update to the Raft state machine.
`/kv/{key}`	GET	Reads a value from the replicated state machine.
`/kv/{key}`	DELETE	Deletes a value from the state machine.
`/health`	GET	Returns the liveness status of the Controller node.
`/status`	GET	Returns technical Raft details (Term, CommitIndex, Role).
`/leader`	GET	Returns the address of the current cluster leader.

Chapter 6

Correctness verification and Fault injection testing

6.1 Methodology for testing distributed systems

As testing distributed systems requires independent servers I considered two options. First option was to run tests using multiple computers. This option however wasn't developer friendly. Every time I would want to test my system I had to boot multiple computers, synchronize data through version control system, then set it up. This would take too much time and effort just to run some basic testing. Thats where the second option came in - I simulated distributed servers using **Docker Containers** (a way to package and run software in an isolated environment). As I can control almost everything about Docker Containers - restarts, failures, networks, this was my way of testing.

6.1.1 Docker-based Simulation Environment

To facilitate consistent and repeatable testing, the entire system is orchestrated using `docker-compose`. The current setup simulates a cluster composed of 3 Controllers and 5 Data Nodes. This configuration is defined in the `docker-compose.yml` file, which manages:

- **Network Isolation:** It defines two distinct bridge networks: `raft-cluster` for internal gRPC communication between Raft nodes, and `client-access` for external API requests.
- **Persistence:** Volumes are mapped for each controller to ensure that Raft logs and snapshots survive container restarts, allowing for testing of recovery mechanisms.
- **Service Dependencies:** Data nodes are configured to depend on controllers,

mirroring the real-world startup sequence where nodes must register with a functional control plane.

This containerized approach allows for complex failure scenarios (like killing a node or partitioning the network via `docker network disconnect`) to be automated in scripts.

6.2 Unit and integration tests for the Raft implementation

As noted in chapter 3, Raft is a central thing in Control Plane, and Control Plane is base for everything in the system. Because of that my Raft algorithm implementation had to be throughoutly tested. I implemented unit tests for every major method in the system with edge cases. In particular I have unit tests of:

- elections - basic check whether it works on 3 and 5 nodes, then tests around replying to various Term value, then Election timeout test
- messages - creation, conversion between messages struct and grpc definitions
- persistence - saving logs, loading logs, comparing whether loaded logs are correct, saving empty logs, saving to invalid paths, saving to corrupted file
- snapshot - whether indexes in log are correct after snapshot, whether snapshot is saved correctly - after load to we have correct data, checking whether logs is truncated after snapshot, test for snapshot installation rpc, checking whether snapshot will run if below threshold or log empty, checking whether snapshot creates with any directory problems

But as far as **unit tests** (tests that check a single small part of the code in isolation) can go, they cannot test how parts of the system behaves in terms of each other. There come **integration tests** (tests that check if different parts of the system work together correctly):

- Full test of election, writing to a log and replication of this log to other nodes
- Test of forwarding to a leader if broadcast is sent to non-leader node
- Test of Raft recovery - whether node will have correct values after it crashes

There was also a **benchmark** (a test used to measure the performance or speed of a system) created for Raft only. It tried to run as many requests per second as possible. On Macbook Air M1 2020, 16GB RAM it achieved around 290k requests per second.

6.3 Simulation of failures and network partitioning

While the Raft tests are useful for testing Raft, they are not useful for testing the entire system. System requirement was to be Strongly Consistent and have Partition Tolerance. To verify these properties, I developed a suite of automated shell scripts located in `tests/fault_injection/` and `tests/e2e/`.

6.3.1 Fault Injection and End-to-End Test Suite

The two primary verification scripts are:

1. `test_network_partition.sh`: This script verifies Partition Tolerance by executing the following sequence:
    - Start the system and wait for it to reach a stable state.
    - Identifies the current Raft leader and disconnects it from the network.
    - Waits for the remaining nodes to detect the failure and elect a new leader.
    - Modifies the cluster topology (e.g., adding a node) via the new leader.
    - Reconnects the original leader and verifies that it successfully synchronizes with the new majority and reflects the changes.
2. `test_strong_consistency.sh`: This script verifies the system's linearizability (strong consistency) by performing multiple sub-tests:
    - First configured some topology of the system to look like it was running for some time to check read-after-write on it. The test run PUT request for some random value to the key and without waiting asked for the value. The expected value of read was the write value.
    - Second was to write sequence of values to the same key (change the key value multiple times). Then there was one singular read sent. The expected value was the value of the last write.
    - Third part was similar to first, but this time there were multiple keys used.
    - Fourth part was about Control Plane consistency, the test tried to register a new node and expected the registration to succeed and be visible in following read
    - Fifth part was to expect all Raft nodes to be available after all of these operations from previous subtests
    - Sixth part was to write a value to some key and restart data-node holding it. The expected outcome was to data-node to have this value in the memory.

- Seventh part was a rerun of second part, to make sure that data-node after restart is still functioning in the intended way.
- Eight, final part was to write a value to data-node and then immediately do a read-after-write check whether replica has this data.

3. `basic_e2e_test.sh`: Located in the `tests/e2e/` directory, this script performs a full lifecycle test: starting the cluster, registering nodes, storing a large set of keys, and verifying they can be retrieved after a series of controlled node restarts.

All of these tests succeeded, guaranteeing that the system is Strongly Consistent and Partition Tolerant.

6.4 Continuous Integration (CI/CD)

To maintain code quality and prevent regressions (new bugs in old code) during development, I implemented a **Continuous Integration** (CI) pipeline—an automated process that builds and tests the code every time a change is made—using **GitHub Actions**. The workflow is defined in `.github/workflows/go.yml` and performs the following steps on every pull request or push to the main branch:

- **Environment Provisioning:** It sets up a Linux environment with the required Go version (currently 1.24).
- **Dependency Caching:** To optimize execution time, the pipeline caches Go modules and build artifacts, typically reducing the build stage duration by over 50%.
- **Automated Testing:** It executes the command `go test ./... --short`. The `--short` flag is used in the CI environment to skip long-running benchmarks or extremely heavy integration tests while still verifying the correctness of all core logic, including Raft state transitions and data node CRUD operations.

This automated feedback loop was essential for the rapid development of complex consensus logic, where a small change in term handling could otherwise have catastrophic effects on cluster stability.

Chapter 7

Performance Analysis and Comparison with Existing Solutions

7.1 Benchmarks

The primary advantage of a cache is its speed. If performance were not a priority, a traditional database would be a more suitable choice. Consequently, a thorough performance analysis is crucial.
I conducted benchmarks to calculate the **Requests Per Second (RPS)** (the throughput or speed) the cache can support. Each benchmark was executed for 10 seconds using 100 workers. The tool issues as many write requests as possible, with each request targeting a unique key. After 10 seconds, the total number of requests is divided by 10 to determine the average RPS.

The benchmarks were performed on a MacBook Air M1 (2020) with 16 GB of RAM, running macOS Tahoe 15.1.

7.1.1 Benchmark Toolset

To ensure objective and reproducible results, I developed a suite of benchmarking tools located in the `tests/benchmark/` directory:

- `benchmark_rps.go`: A high-performance Go-based benchmark that measures maximum RPS using goroutines and connection pooling.
- `benchmark_scaling.sh`: Orchestrates tests across multiple cluster configurations to measure **horizontal scaling** (adding more servers to increase performance) efficiency.

- `benchmark_raft_delay.sh`: Evaluates the impact of consensus latency on Control Plane performance by injecting artificial Raft processing delays.
- `benchmark_comparison.sh`: Automates the deployment of both this system and competitor systems (Etcd, Redis) to provide direct side-by-side performance comparisons.

7.2 Throughput Study (RPS) Depending on the Number of Clients and Shards

Table 7.1: Client Scaling (One DataNode)

Workers	RPS
1	5,714
10	18,855
50	28,336
100	29,605
200	27,661

Table 7.2: Horizontal Scaling (Multiple DataNodes, No Replication)

DataNodes	Combined RPS	Scaling
1	28,460	1.0x
2	55,001	1.93x
3	72,731	2.56x

Table 7.3: Replication Impact (Per-node Throughput)

Configuration	RPS
1 DataNode (no replication)	28,909
2 DataNodes (no replication)	27,100
3 DataNodes (no replication)	27,468
2 DataNodes (WITH replication)	26,108
3 DataNodes (WITH replication)	27,657

7.3 Analysis of Raft Protocol Overhead on Latency

Latency is the delay between a user's request and the server's response. I created a benchmark for a case where Raft was used for every single operation. In a standard deployment, this is not necessary because the Smart Client hashes the topology and does not need to consult the Control Plane for every request.
However, in this specific benchmark, I forced the client to send every request through

the Raft consensus module. This resulted in approximately 8,000 requests per second. I then implemented an artificial delay for the broadcast and *LogRequest* RPC operations, allowing the system to simulate varying network or processing latencies. This demonstrates the extent to which Raft performance influences the overall system. The results were as follows:

Table 7.4: Analysis of Raft Protocol Overhead on Response Time

Artificial Delay	Consensus RPS	Impact
0 ms	8002	Baseline
10 ms	274	29x slower
25 ms	100	80x slower
50 ms	59	135x slower

This clearly illustrates the significant impact that Raft consensus speed has on the overall system performance.

7.4 Comparative Study

To evaluate the performance of my implementation, it is necessary to compare it against established solutions. I identified two primary open-source systems for comparison, based on current industry popularity [7]. The first is Etcd, which is also a Raft-based distributed key-value store that guarantees strong consistency. However, Etcd is a durable store, meaning data is persisted to disk to survive node crashes. The second system is Redis, the most widely used cache in the world [8, 7]. Redis is primarily an AP (Availability and Partition Tolerance) system, meaning it prioritizes performance over strong consistency, offering eventual consistency instead. Redis is a highly optimized system written in C with over 15 years of development. Matching the performance of Redis is a significant challenge, making it an excellent benchmark for speed.

7.4.1 Performance Comparison with Etcd

Etcd was deployed using Docker. I utilized the official client package `go.etcd.io/etcd/client/v3@v3`. I performed ten iterations of the 10-second, 100-worker benchmark described in Section 7.1. The results were averaged, yielding a score of 26,661 RPS.

7.4.2 Performance Comparison with Redis

Redis was also deployed using Docker, and the official `github.com/redis/go-redis/v9` client was used. Following the same methodology as with Etcd, the averaged result over ten runs was 38,776 RPS.

7.4.3 Conclusions from the Comparative Analysis

I re-ran my system alongside Etcd and Redis using the same 10x10-second, 100-worker configuration. My implementation averaged 27,715 RPS, making it approximately 3.95% faster than Etcd and 30% slower than Redis.
These results are consistent with expectations. My system is faster than Etcd, likely due to the lack of disk persistence in the Data Plane, but slower than Redis due to the latter's highly optimized C implementation, more lenient consistency guarantees, and extensive performance tuning. Further optimizations to the Raft implementation could close the gap, but reaching Redis-level performance would be extremely difficult given the architectural trade-offs required for strong consistency.

Chapter 8

Summary

8.1 Evaluation of the Achievement of Research Objectives

The functional and non-functional requirements of the system have all been met. Every requirement was verified with a corresponding test case to ensure correctness.

8.1.1 Encountered Implementation Problems and Challenges

The most significant challenge of the project was the implementation itself. Since the original Raft paper provides high-level algorithmic details rather than low-level implementation specifications, designing the concurrency model and operational logic was my primary responsibility.
Initially, I implemented Raft using multiple mutexes, which made the system difficult to reason about and led to several deadlocks that required extensive debugging.
Another challenge was the delayed implementation of snapshotting logic. This required a significant refactoring of the log indexing and term management to correctly handle truncated logs.
Furthermore, the initial implementation of snapshots was inefficient. Sending a large snapshot in a single request blocked the main thread and held the state mutex for too long, making the leader unresponsive to heartbeats. This caused a bug where another node would detect a leader failure and initiate a new election during the transfer. To resolve this, I implemented a chunked transfer protocol, which allowed the leader to remain responsive while synchronizing followers.

8.2 Directions for Further Development

As discussed in Chapter 7, the performance of the Raft consensus module is the primary bottleneck for the entire application. I have identified several potential areas for further improvement:

- **Batching Requests:** Currently, log entries are persisted for every request. Performance could be significantly improved by batching multiple entries into a single I/O operation. This would reduce the frequency of file open and close operations. However, this must be implemented carefully; if the system acknowledges a request before the batch is safely persisted to stable storage, it would violate strong consistency guarantees.
- **I/O Optimization:** Disk I/O is a slow operation. The system could benefit from keeping log files open based on a heuristic to avoid repeated overhead. Frequent `fsync()` calls are necessary for reliability, but a more sophisticated file management strategy could bridge the performance gap without jeopardizing log persistence.
- **Encoding Performance:** Profiling reveals that binary encoding and decoding account for approximately 40% of the application's processing time. Implementing a custom, more efficient serialization format could significantly reduce this overhead.
- **Concurrency Tuning:** The system might benefit from a more optimized thread-to-core mapping for handle-intensive I/O operations.

Bibliography

[1] E. Brewer, "CAP twelve years later: How the "rules" have changed" in *Computer*, vol. 45, no. 2, Feb. 2012.

[2] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm" in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, 2014.

[3] T. J. Edwards et al., "CockroachDB: The Resilient Geo-Distributed SQL Database" in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020.

[4] S. Zhou and S. Mu, "Fault-Tolerant Replication with Pull-Based Consensus in MongoDB" in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021. [Online]. Available: https://www.usenix.org/conference/nsdi21/presentation/zhou. [Accessed: 21-Dec-2025].

[5] Neo4j, "Causal Clustering Architecture" [Online]. Available: https://neo4j.com/docs/operations-manual/current/clustering/introduction/. [Accessed: 20-Dec-2025].

[6] RabbitMQ, "Quorum Queues" [Online]. Available: https://www.rabbitmq.com/docs/quorum-queues. [Accessed: 20-Dec-2025].

[7] DB-Engines, "Ranking of Key-Value Stores" [Online]. Available: https://db-engines.com/en/ranking/key-value+store. [Accessed: 20-Dec-2025].

[8] Redis, "Get Started with Redis" [Online]. Available: https://redis.io/docs/latest/get-started/. [Accessed: 21-Dec-2025].

[9] D. Karger et al., "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web" in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, 1997.

[10] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store" in *SOSP '07: Proceedings of the twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007.

[11] TiKV Project, "TiKV: Deep dive introduction" [Online]. Available: https://tikv.org/deep-dive/introduction/. [Accessed: 21-Dec-2025].

[12] G. Fowler, L. C. Noll, and K. Vo, "The FNV Non-Cryptographic Hash Algorithm" [Online]. Available: https://tools.ietf.org/html/draft-eastlake-fnv-17. [Accessed: 21-Dec-2025].

[13] Apache Cassandra, "Dataset Partitioning: Consistent Hashing" [Online]. Available: https://cassandra.apache.org/doc/5.0/cassandra/architecture/dynamo.html#dataset-partitioning- [Accessed: 21-Dec-2025].

[14] L. Lamport, "The Part-Time Parliament" in *ACM Transactions on Computer Systems*, vol. 16, no. 2, May 1998.

[15] L. Lamport, "Paxos Made Simple" in *ACM SIGACT News (Distributed Computing Column)*, vol. 32, no. 4, Dec. 2001.